Numpy 2D array :

- its a combination of 1D array.
- Represent as [[ ]]

In [1]:
```
import numpy as np
```

In [2]:
```
a= np.array([[0,1,2,3] ,[5,6,8,9]])
print(a)
```

```
[[0 1 2 3]
 [5 6 8 9]]
```

In [3]:
```
a.ndim
```

Out[3]: 2

In [4]:
```
a.shape
```

Out[4]: (2, 4)

In [5]:
```
a= np.zeros((3,3))
print(a)
```

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

In [6]:
```
a= np.zeros((3,3),dtype=int)
print(a)
```

```
[[0 0 0]
 [0 0 0]
 [0 0 0]]
```

In [7]:
```
a= np.ones((3,3))
print(a)
```

```
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

In [8]:
```
a= np.ones((3,3),dtype=int)
print(a)
```

```
[[1 1 1]
 [1 1 1]
 [1 1 1]]
```

In [9]:
```
a=np.eye(3,3)  #identity matrix
print(a)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

In [10]:
```
a= np.diag([1,2,3])  #diagonal matrix
print(a)
```

```
[[1 0 0]
 [0 2 0]
 [0 0 3]]
```

In [11]:
```
a= np.array([[0,1,2,3] ,[5,6,8,9],[10,11,12,13]]) #output as diagonal element
print(a)
```

```
[[ 0  1  2  3]
 [ 5  6  8  9]
 [10 11 12 13]]
```

In [12]:
```
print(np.diag(a))
```

```
[ 0  6 12]
```

In [13]:
```
a= np.array([[0,1,2,] ,[5,6,8,],[10,11,12,]])
print(a)
```

```
[[ 0  1  2]
 [ 5  6  8]
 [10 11 12]]
```

print(np.diag(a)) #output as diagonal element

indexing in 2D array

In [14]:
```
a=np.arange(10,91,10).reshape(3,3)
print(a)
```

```
[[10 20 30]
 [40 50 60]
 [70 80 90]]
```

In [15]:
```
print(a[0])   # 0th row
```

```
[10 20 30]
```

In [16]:
```
print(a[1])    # 1st row
```

```
[40 50 60]
```

In [17]:
```
print(a[2])   #2nd row
```

```
[70 80 90]
```

In [18]:
```
print(a[:,0]) #0th colounm
```

```
[10 40 70]
```

In [19]:
```
print(a[:,[1,2]]) #1st,2nd colounm
```

```
[[20 30]
 [50 60]
 [80 90]]
```

In [20]:
```
a[0:3]    # [str row index:(n-1) row index]
           # [:] fetching only rows
```

Out[20]:
```
array([[10, 20, 30],
       [40, 50, 60],
       [70, 80, 90]])
```

In [21]:
```
a[1:3]
```

Out[21]:
```
array([[40, 50, 60],
       [70, 80, 90]])
```

fetching specific value of matrix

[row , colounm]

In [22]:
```
print(a)
```

```
[[10 20 30]
 [40 50 60]
 [70 80 90]]
```

In [23]:
```
print(a[1,2])
```

```
60
```

In [24]:
```
print(a[1,1])
```

```
50
```

assigning value in 2D array

In [25]:
```
print(a)
```

```
[[10 20 30]
 [40 50 60]
 [70 80 90]]
```

In [26]:
```
a[0] = [11,21,31]
print(a)
```

```
[[11 21 31]
 [40 50 60]
 [70 80 90]]
```

In [27]:
```
a[1,1]=51
print(a)
```

```
[[11 21 31]
 [40 51 60]
 [70 80 90]]
```

flattening : converting nd array to 1D array

ravel()

In [28]:
```
print(a)
```

```
[[11 21 31]
 [40 51 60]
 [70 80 90]]
```

In [29]:
```
b=a.ravel()
print(b)
```

```
[11 21 31 40 51 60 70 80 90]
```

reshape and transpose(T)

In [30]:
```
a=np.arange(5,11).reshape(3,2)
print(a)
```

```
[[ 5  6]
 [ 7  8]
 [ 9 10]]
```

In [31]:
```
b=a.T         #T = row convert colom or colom convert row
print(b)
```

```
[[ 5  7  9]
 [ 6  8 10]]
```

sorting: along an axis=1 (row), axis=0(colom)

note:in pandas it will be opposite axis 1> colom,axis 0> row

In [32]:
```
a=np.array([[5,4,6],[2,8,2],[8,9,20]])
print(a)
```

```
[[ 5  4  6]
 [ 2  8  2]
 [ 8  9 20]]
```

In [33]:
```
b=np.sort(a,axis=0)
print(b)
```

```
[[ 2  4  2]
 [ 5  8  6]
 [ 8  9 20]]
```

In [34]:
```
b=np.sort(a,axis=1)
print(b)
```

```
[[ 4  5  6]
 [ 2  2  8]
 [ 8  9 20]]
```

adding and dot product(a*b) :

In [35]:
```
print(a)
```

```
[[ 5  4  6]
 [ 2  8  2]
 [ 8  9 20]]
```

In [36]:
```
print(b)
```

```
[[ 4  5  6]
 [ 2  2  8]
 [ 8  9 20]]
```

In [37]:
```
print(a+b)
```

```
[[ 9  9 12]
 [ 4 10 10]
 [16 18 40]]
```

In [38]:
```
print(a*b)
```

```
[[ 20  20  36]
 [  4  16  16]
 [ 64  81 400]]
```

vertical stack

In [40]:
```
print(a)
print(b)
```

```
[[ 5  4  6]
 [ 2  8  2]
 [ 8  9 20]]
[[ 4  5  6]
 [ 2  2  8]
 [ 8  9 20]]
```

In [41]:
```
np.vstack((a,b))
```

Out[41]:
```
array([[ 5,  4,  6],
       [ 2,  8,  2],
       [ 8,  9, 20],
       [ 4,  5,  6],
       [ 2,  2,  8],
       [ 8,  9, 20]])
```

In [42]:
```
np.concatenate((a,b),axis=0)   #concatenate = vstack
```

Out[42]:
```
array([[ 5,  4,  6],
       [ 2,  8,  2],
       [ 8,  9, 20],
       [ 4,  5,  6],
       [ 2,  2,  8],
       [ 8,  9, 20]])
```

In [43]:
```
np.hstack((a,b))
```

Out[43]:
```
array([[ 5,  4,  6,  4,  5,  6],
       [ 2,  8,  2,  2,  2,  8],
       [ 8,  9, 20,  8,  9, 20]])
```

In [45]:
```
np.concatenate((a,b),axis=1)   #concatenate = hstack
```

Out[45]:
```
array([[ 5,  4,  6,  4,  5,  6],
       [ 2,  8,  2,  2,  2,  8],
       [ 8,  9, 20,  8,  9, 20]])
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: